

Sec. 4.2. Windows PowerShell

Windows PowerShell

Task 1 - What Is PowerShell

PowerShell, objects are fundamental units that encapsulate data and functionality, making it easier to manage and manipulate information. An object in PowerShell can contain file names, usernames or sizes as data (**properties**), and carry functions (**methods**) such as copying a file or stopping a process.

Answer the questions below

What do we call the advanced approach used to develop PowerShell? object-oriented

Task 2 - PowerShell Basics

PowerShell can be launched in several ways, depending on your needs and environment. If you are working on a Windows system from the graphical interface (GUI), these are some of the possible ways to launch it:

- **Start Menu:** Type `powershell` in the Windows Start Menu search bar, then click on `Windows PowerShell` or `PowerShell` from the results.
- **Run Dialog:** Press `Win + R` to open the `Run` dialog, type `powershell`, and hit `Enter`.
- **File Explorer:** Navigate to any folder, then type `powershell` in the address bar, and press `Enter`. This opens PowerShell in that specific directory.
- **Task Manager:** Open the Task Manager, go to `File > Run new task`, type `powershell`, and press `Enter`.

Alternatively, PowerShell can be launched from a Command Prompt (`cmd.exe`) by typing `powershell`, and pressing `Enter`. After PowerShell has launched, we're presented with a `PS` (which stands for `PowerShell`) prompt in the current working directory.

PowerShell commands are known as `cmdlets` (pronounced `command-lets`). They are much more powerful than the traditional Windows commands and allow for more

advanced data manipulation.

Cmdlets follow a consistent **Verb-Noun** naming convention. This structure makes it easy to understand what each cmdlet does. The **Verb** describes the action, and the **Noun** specifies the object on which action is performed. For example:

- **Get-Content** : Retrieves (gets) the content of a file and displays it in the console.
- **Set-Location** : Changes (sets) the current working directory.

to list all available cmdlets, functions, aliases, and scripts that can be executed in the current PowerShell session, we can use **Get-Command**.

```
PS C:\Users\captain> Get-Command
```

CommandType	Name
Version	Source
-----	----
-----	-----
Alias	Add-AppPackage
2.0.1.0	Appx
Alias	Add-AppPackageVolume
2.0.1.0	Appx
Alias	Add-AppProvisionedPackage
3.0	Dism
[...]	
Function	A:
Function	Add-BCDataCacheExtension
1.0.0.0	BranchCache
Function	Add-DnsClientDohServerAddress
1.0.0.0	DnsClient
[...]	
Cmdlet	Add-AppxPackage
2.0.1.0	Appx
Cmdlet	Add-AppxProvisionedPackage
3.0	Dism
Cmdlet	Add-AppxVolume

```
2.0.1.0    Appx
[...]
```

For each `CommandInfo` object retrieved by the cmdlet, some essential information (properties) is displayed on the console. For example, if we want to display only the available commands of type "function", we can use `-CommandType "Function"`, as shown below:

```
PS C:\Users\captain> Get-Command -CommandType "Function"
```

CommandType	Name
Version	Source
-----	----
-----	-----
Function	A:
Function	Add-BCDataCacheExtension
1.0.0.0	BranchCache
Function	Add-DnsClientDohServerAddress
1.0.0.0	DnsClient
Function	Add-DnsClientNrptRule
1.0.0.0	DnsClient
[...]	

`Get-Help`: it provides detailed information about cmdlets, including usage, parameters, and examples.

```
PS C:\Users\captain> Get-Help Get-Date
```

NAME

Get-Date

SYNOPSIS

Gets the current date and time.

SYNTAX

```
Get-Date [[-Date] <System.DateTime>] [-Day <System.Int32>]
[-DisplayHint {Date | Time | DateTime}] [-Format <System.String>]
```

```
>] [-Hour <System.Int32>] [-Millisecond <System.Int32>] [-Minute <System.Int32>] [-Month <System.Int32>] [-Second <System.Int32>] [-Year <System.Int32>] [<CommonParameters>]
```

```
Get-Date [[-Date] <System.DateTime>] [-Day <System.Int32>] [-DisplayHint {Date | Time | DateTime}] [-Hour <System.Int32>] [-Millisecond <System.Int32>] [-Minute <System.Int32>] [-Month <System.Int32>] [-Second <System.Int32>] [-UFormat <System.String>] [-Year <System.Int32>] [<CommonParameters>]
```

DESCRIPTION

The ``Get-Date`` cmdlet gets a `DateTime` object that represents the current date or a date that you specify. ``Get-Date`` can format the date and time in several .NET and UNIX formats. You can use ``Get-Date`` to generate a date or time character string, and then send the string to other cmdlets or programs.

``Get-Date`` uses the current culture settings of the operating system to determine how the output is formatted. To view your computer's settings, use ``(Get-Culture).DateTimeFormat``.

RELATED LINKS

Online Version: https://learn.microsoft.com/powershell/module/microsoft.powershell.utility/get-date?view=powershell-5.1&WT.mc_id=ps-gethelp

`ForEach-Object`

`Get-Culture`

`Get-Member`

`New-Item`

`New-TimeSpan`

`Set-Date`

`Set-Culture` `xref:International.Set-Culture`

REMARKS

To see the examples, type: `"get-help Get-Date -examples"`.

For more information, type: `"get-help Get-Date -detailed"`.

For technical information, type: "get-help Get-Date -full".
For online help, type: "get-help Get-Date -online".

As shown in the results above, `Get-Help` informs us that we can retrieve other useful information about a cmdlet by appending some options to the basic syntax. For example, by appending `-examples` to the command displayed above, we will be shown a list of common ways in which the chosen cmdlet can be used.

To make the transition easier for IT professionals, PowerShell includes aliases — which are shortcuts or alternative names for cmdlets— for many traditional Windows commands. Indispensable for users already familiar with other command-line tools, `Get-Alias` lists all aliases available. For example, `dir` is an alias for `Get-ChildItem`, and `cd` is an alias for `Set-Location`.

```
PS C:\Users\captain> Get-Alias
```

CommandType	Name
Version	Source
-----	----
-----	-----
Alias	% -> ForEach-Object
Alias	? -> Where-Object
Alias	ac -> Add-Content
Alias	asnp -> Add-PSSnapin
Alias	cat -> Get-Content
Alias	cd -> Set-Location
Alias	CFS -> ConvertFrom-String
3.1.0.0	Microsoft.PowerShell.Utility
Alias	chdir -> Set-Location
Alias	clc -> Clear-Content
Alias	clear -> Clear-Host
[...]	

To search for modules (collections of cmdlets) in online repositories like the PowerShell Gallery, we can use `Find-Module`. Sometimes, if we don't know the exact name of the module, it can be useful to search for modules with a similar name. We can achieve this by filtering the `Name` property and appending a wildcard

(*) to the module's partial name, using the following standard PowerShell syntax: `Cmdlet -Property "pattern*"`.

```
PS C:\Users\captain> Find-Module -Name "PowerShell*"
```

Version	Name	Repository
0.4.7	powershell-yaml	PSGallery
Powershell module for serializing and deserializing YAML		
2.2.5	PowerShellGet	PSGallery
PowerShell module with commands for discovering, installing, updating and publishing the PowerShell artifacts like Modules, DSC Resources, Role Capabilities and Scripts.		
1.0.80.0	PowerShell.Module.InvokeWinGet	PSGallery
Module to Invoke WinGet and parse the output in PSObjects		
0.17.0	PowerShellForGitHub	PSGallery
PowerShell wrapper for GitHub API		

Once identified, the modules can be downloaded and installed from the repository with `Install-Module`, making new cmdlets contained in the module available for use.

```
PS C:\Users\captain> Install-Module -Name "PowerShellGet"
```

Untrusted repository

You are installing the modules from an untrusted repository. If you trust this repository, change its InstallationPolicy value by running the Set-PSRepository cmdlet. Are you sure you want to install the modules from 'PSGallery'?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend
[?] Help (default is "N"):

Answer the questions below

How would you retrieve a list of commands that start with the verb Remove? [for the sake of this question, avoid the use of quotes (" or ') in your answer] `Get-Command -Name Remove*`

What cmdlet has its traditional counterpart echo as an alias? `Write-Output`

What is the command to retrieve some example usage for the cmdlet `New-LocalUser`? `Get-Help New-LocalUser -examples`

Task 3 - Navigating the File System and Working with Files

`dir` command in Command Prompt (or `ls` in Unix-like systems), `Get-ChildItem` lists the files and directories in a location specified with the `-Path` parameter. It can be used to explore directories and view their contents. If no `Path` is specified, the cmdlet will display the content of the current working directory.

```
PS C:\Users\captain> Get-ChildItem
```

Directory: C:\Users\captain

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-r---	5/8/2021	9:15 AM		Desktop
d-r---	9/4/2024	10:58 AM		Documents
d-r---	5/8/2021	9:15 AM		Downloads
d-r---	5/8/2021	9:15 AM		Favorites
d-r---	5/8/2021	9:15 AM		Links
d-r---	5/8/2021	9:15 AM		Music
d-r---	5/8/2021	9:15 AM		Pictures
d-----	5/8/2021	9:15 AM		Saved Games
d-r---	5/8/2021	9:15 AM		Videos

To navigate to a different directory, we can use the `Set-Location` cmdlet. It changes the current directory, bringing us to the specified path, akin to the `cd` command in Command Prompt.

```
PS C:\Users\captain> Set-Location -Path ".\Documents"
PS C:\Users\captain\Documents>
```

To create an item in PowerShell, we can use `New-Item`. We will need to specify the path of the item and its type (whether it is a file or a directory).

```
PS C:\Users\captain\Documents> New-Item -Path ".\captain-cabin
\captain-wardrobe" -ItemType "Directory"
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
d----	9/4/2024 12:20 PM		captain-wardrobe

```
PS C:\Users\captain\Documents> New-Item -Path ".\captain-cabin
\captain-wardrobe\captain-boots.txt" -ItemType "File"
```

Directory: C:\Users\captain\Documents\captain-cabin\captain-wardrobe

Mode	LastWriteTime	Length	Name
-a----	9/4/2024 11:46 AM	0	captain-boots.txt

Similarly, the `Remove-Item` cmdlet removes both directories and files, whereas in Windows CLI we have separate commands `rmdir` and `del`.

```
PS C:\Users\captain\Documents> Remove-Item -Path ".\captain-cabin
\captain-wardrobe\captain-boots.txt"
PS C:\Users\captain\Documents> Remove-Item -Path ".\captain-cabin
\captain-wardrobe"
```

We can copy or move files and directories alike, using respectively `Copy-Item` (equivalent to `copy`) and `Move-Item` (equivalent to `move`).

```
PS C:\Users\captain\Documents> Copy-Item -Path .\captain-cabin
\captain-hat.txt -Destination .\captain-cabin\captain-hat2.txt
PS C:\Users\captain\Documents> Get-ChildItem -Path ".\captain-cabin\"
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
d-----	9/4/2024 12:50 PM		captain-wardrobe
-a----	9/4/2024 12:50 PM	0	captain-boots.txt
-a----	9/4/2024 12:14 PM	264	captain-hat.txt
-a----	9/4/2024 12:14 PM	264	captain-hat2.txt
-a----	9/4/2024 12:37 PM	2116	ship-flag.txt

Finally, to read and display the contents of a file, we can use the `Get-Content` cmdlet, which works similarly to the `type` command in Command Prompt (or `cat` in Unix-like systems).

```
PS C:\Users\captain\Documents\captain-cabin> Get-Content -Path
".\captain-hat.txt"
```

```
 _ _
| |   | | | |
| |__ | |
| ' \ / | |
| | | ( | |
|_| |_| \_,_| \_|
```

Don't touch my hat!

Answer the questions below

What cmdlet can you use instead of the traditional Windows command type? Get-Content

What PowerShell command would you use to display the content of the "C:\Users" directory? [for the sake of this question, avoid the use of quotes (" or ') in your answer] Get-ChildItem -Path C:\Users

How many items are displayed by the command described in the previous question? 4

Task 4 - Piping, Filtering, and Sorting Data

Piping is a technique used in command-line environments that allows the output of one command to be used as the input for another. This creates a sequence of operations where the data flows from one command to the next. Represented by the `|` symbol, piping is widely used in the Windows CLI, as introduced earlier in this module, as well as in Unix-based shells. In PowerShell, piping is even more powerful because it passes **objects** rather than just text. These objects carry not only the data but also the properties and methods that describe and interact with the data. For example, if you want to get a list of files in a directory and then sort them by size, you could use the following command in PowerShell:

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Sort-Object Length
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	9/4/2024 12:50 PM	0	captain-boot
s.txt			
-a----	9/4/2024 12:14 PM	264	captain-hat2.
txt			
-a----	9/4/2024 12:14 PM	264	captain-hat.t
xt			
-a----	9/4/2024 12:37 PM	2116	ship-flag.txt

```
d-----          9/4/2024  12:50 PM          captain-wardr
obe
```

Here, `Get-ChildItem` retrieves the files (as objects), and the pipe (`|`) sends those file objects to `Sort-Object`, which then sorts them by their `Length` (size) property. This object-based approach allows for more detailed and flexible command sequences. In the example above, we have leveraged the `Sort-Object` cmdlet to sort objects based on specified properties. Beyond sorting, PowerShell provides a set of cmdlets that, when combined with piping, allow for advanced data manipulation and analysis. To filter objects based on specified conditions, returning only those that meet the criteria, we can use the `Where-Object` cmdlet. For instance, to list only `.txt` files in a directory, we can use:

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Where-Object -Property "Extension" -eq ".txt"
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	9/4/2024 12:50 PM	0	captain-boots.txt
-a----	9/4/2024 12:14 PM	264	captain-hat.txt
-a----	9/4/2024 12:14 PM	264	captain-hat2.txt
-a----	9/4/2024 12:37 PM	2116	ship-flag.txt

Here, `Where-Object` filters the files by their `Extension` property, ensuring that only files with extension equal (`-eq`) to `.txt` are listed.

The operator `-eq` (i.e. "**equal to**") is part of a set of **comparison operators** that are shared with other scripting languages (e.g. Bash, Python). To show the potentiality of the PowerShell's filtering, we have selected some of the most useful operators from that list:

- `ne`: "**not equal**". This operator can be used to exclude objects from the results based on specified criteria.

- **gt**: "**greater than**". This operator will filter only objects which exceed a specified value. It is important to note that this is a strict comparison, meaning that objects that are equal to the specified value will be excluded from the results.
- **ge**: "**greater than or equal to**". This is the non-strict version of the previous operator. A combination of **gt** and **eq**.
- **lt**: "**less than**". Like its counterpart, "greater than", this is a strict operator. It will include only objects which are strictly below a certain value.
- **le**: "**less than or equal to**". Just like its counterpart **ge**, this is the non-strict version of the previous operator. A combination of **lt** and **eq**.

Below, another example shows that objects can also be filtered by selecting properties that match (**-like**) a specified pattern:

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Where-Object -Property "Name" -like "ship*"
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	9/4/2024 12:37 PM	2116	ship-flag.txt

The next filtering cmdlet, **Select-Object**, is used to select specific properties from objects or limit the number of objects returned. It's useful for refining the output to show only the details one needs.

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Select-Object Name,Length
```

Name	Length
----	-----
captain-wardrobe	
captain-boots.txt	0
captain-hat.txt	264

```
captain-hat2.txt 264
ship-flag.txt    2116
```

The cmdlets pipeline can be extended by adding more commands, as the feature isn't limited to just piping between two cmdlets. As an exercise, try and build a pipeline of cmdlets to sort and filter the output with the goal of displaying the largest file in the `C:\Users\captain\Documents\captain-cabin` directory.

```
Get-ChildItem | Sort-Object Length -Descending | Select-Object
-First 1
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	9/4/2024 12:37 PM	2116	ship-flag.txt

The last in this set of filtering cmdlets is `Select-String`. This cmdlet searches for text patterns within files, similar to `grep` in Unix-based systems or `findstr` in Windows Command Prompt. It's commonly used for finding specific content within log files or documents.

```
PS C:\Users\captain\Documents\captain-cabin> Select-String -Pat
h ".\captain-hat.txt" -Pattern "hat"
```

```
captain-hat.txt:8:Don't touch my hat!
```

The `Select-String` cmdlet fully supports the use of regular expressions ([regex](#)([opens in new tab](#))). This advanced feature allows for complex pattern matching within files, making it a powerful tool for searching and analysing text data.

Answer the questions below

How would you retrieve the items in the current directory with size greater than 100? [for the sake of this question, avoid the use of quotes (" or ') in your answer]

```
Get-ChildItem | Where-Object -Property Length -gt 100
```

Task 5 - System and Network Information

The `Get-ComputerInfo` cmdlet retrieves comprehensive system information, including operating system information, hardware specifications, BIOS details, and more. It provides a snapshot of the entire system configuration in a single command. Its traditional counterpart `systeminfo` retrieves only a small set of the same details.

```
PS C:\Users\captain> Get-ComputerInfo

WindowsBuildLabEx                        : 2034
8.859.amd64fre.fe_release_svc_prod2.220707-1832
WindowsCurrentVersion                    : 6.3
WindowsEditionId                         : Serve
rDatacenter
WindowsInstallationType                  : Serve
r Core
WindowsInstallDateFromRegistry           : 4/23/
2024 6:36:29 PM
WindowsProductId                         : 00454
-60000-00001-AA763
WindowsProductName                       : Windo
ws Server 2022 Datacenter
[...]
```

Essential for managing user accounts and understanding the machine's security configuration, `Get-LocalUser` lists all the local user accounts on the system. The default output displays, for each user, username, account status, and description.

```
PS C:\Users\captain> Get-LocalUser

Name                Enabled Description
----                -
Administrator       True    Built-in account for administering t
he computer/domain
captain              True    The beloved captain of this pirate s
hip.
DefaultAccount       False   A user account managed by the syste
m.
Guest                False   Built-in account for guest access to
```

the computer/domain

WDAGUtilityAccount False A user account managed and used by the system for Windows Defender Application Guard scenarios.

Similar to the traditional `ipconfig` command, the following two cmdlets can be used to retrieve detailed information about the system's network configuration. `Get-NetIPConfiguration` provides detailed information about the network interfaces on the system, including IP addresses, DNS servers, and gateway configurations.

```
PS C:\Users\captain> Get-NetIPConfiguration
```

```
InterfaceAlias      : Ethernet
InterfaceIndex      : 5
InterfaceDescription : Amazon Elastic Network Adapter
NetProfile.Name      : Network 3
IPv4Address          : 10.10.178.209
IPv6DefaultGateway  :
IPv4DefaultGateway  : 10.10.0.1
DNSServer            : 10.0.0.2
```

In case we need specific details about the IP addresses assigned to the network interfaces, the `Get-NetIPAddress` cmdlet will show details for all IP addresses configured on the system, including those that are not currently active.

```
PS C:\Users\captain> Get-NetIPAddress
```

```
IPAddress           : fe80::3fef:360c:304:64e%5
InterfaceIndex      : 5
InterfaceAlias       : Ethernet
AddressFamily        : IPv6
Type                 : Unicast
PrefixLength         : 64
PrefixOrigin         : WellKnown
SuffixOrigin         : Link
AddressState         : Preferred
ValidLifetime        : Infinite ([TimeSpan]::MaxValue)
PreferredLifetime    : Infinite ([TimeSpan]::MaxValue)
```

```
SkipAsSource      : False
PolicyStore       : ActiveStore

IPAddress         : ::1
InterfaceIndex    : 1
InterfaceAlias    : Loopback Pseudo-Interface 1
AddressFamily     : IPv6
[...]

IPAddress         : 10.10.178.209
InterfaceIndex    : 5
InterfaceAlias    : Ethernet
AddressFamily     : IPv4
[...]

IPAddress         : 127.0.0.1
InterfaceIndex    : 1
InterfaceAlias    : Loopback Pseudo-Interface 1
AddressFamily     : IPv4
[...]
```

Answer the questions below

Other than your current user and the default "Administrator" account, what other user is enabled on the lab machine? p1r4t3

This lad has hidden his account among the others with no regard for our beloved captain! What is the motto he has so bluntly put as his account's description? A merry life and a short one.

Now a small challenge to put it all together. This shady lad that we just found hidden among the local users has his own home folder in the "C:\Users" directory. Can you navigate the filesystem and find the hidden treasure inside this pirate's home? THM{p34rIlnAsh3I}

Task 6 - Real-Time System Analysis

`Get-Process` provides a detailed view of all currently running processes, including CPU and memory usage, making it a powerful tool for monitoring and troubleshooting.

```
PS C:\Users\captain> Get-Process
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
67	5	872	500	0.06	2340	0	AggregatorHost
55	5	712	2672	0.02	3024	0	AM_De
309	13	18312	1256	0.52	1524	0	lta_Patch_1.417.483.0
							amazon-ssm-agent
78	6	4440	944	0.02	516	0	cmd
94	7	1224	1744	0.31	568	0	conhost
[...]							

`Get-Service` allows the retrieval of information about the status of services on the machine, such as which services are running, stopped, or paused. It is used extensively in troubleshooting by system administrators, but also by forensics analysts hunting for anomalous services installed on the system.

```
PS C:\Users\captain> Get-Service
```

Status	Name	DisplayName
-----	-----	-----
Stopped	Amazon EC2Launch	Amazon EC2Launch
Running	AmazonSSMAgent	Amazon SSM Agent
Stopped	AppIDSvc	Application Identity
Running	BFE	Base Filtering Engine
Running	CertPropSvc	Certificate Propagation

```
Stopped ClipSVC Client License Service (ClipSVC)
[...]
```

To monitor active network connections, `Get-NetTCPConnection` displays current TCP connections, giving insights into both local and remote endpoints. This cmdlet is particularly handy during an incident response or malware analysis task, as it can uncover hidden backdoors or established connections towards an attacker-controlled server.

```
PS C:\Users\captain> Get-NetTCPConnection
```

LocalAddress	LocalPort	RemoteAddress	RemotePort	State
AppliedSetting	OwningProcess			
----	-----	-----	-----	--
---	-----	-----		
[...]				
::	22	::	0	Li
sten	1444			
10.10.178.209	49695	199.232.26.172	80	Ti
meWait	0			
0.0.0.0	49668	0.0.0.0	0	Li
sten	424			
0.0.0.0	49667	0.0.0.0	0	Li
sten	652			
0.0.0.0	49666	0.0.0.0	0	Li
sten	388			
0.0.0.0	49665	0.0.0.0	0	Li
sten	560			
0.0.0.0	49664	0.0.0.0	0	Li
sten	672			
0.0.0.0	3389	0.0.0.0	0	Li
sten	980			
10.10.178.209	139	0.0.0.0	0	Li
sten	4			
0.0.0.0	135	0.0.0.0	0	Li
sten	908			
10.10.178.209	22	10.14.87.60	53523	Es

established Internet	1444			
0.0.0.0	22	0.0.0.0	0	Li
sten	1444			

Additionally, we are going to mention `Get-FileHash` as a useful cmdlet for generating file hashes, which is particularly valuable in incident response, threat hunting, and malware analysis, as it helps verify file integrity and detect potential tampering.

```
PS C:\Users\captain\Documents\captain-cabin> Get-FileHash -Path
.\ship-flag.txt
```

Algorithm	Hash	Path
-----	----	----
SHA256	54D2EC3C12BF3D[...]	C:\Users\captain\Documents\captain-cabin\ship-flag.txt

```
PS C:\Users\Alex\Documents> Get-Item -Path "C:\House\house_log.
txt" -Stream *
```

```
PSPath      : Microsoft.PowerShell.Core\FileSystem::C:\House
\house_log.txt::$DATA
PSParentPath : Microsoft.PowerShell.Core\FileSystem::C:\House
PSChildName  : house_log.txt::$DATA
PSDrive      : C
PSProvider   : Microsoft.PowerShell.Core\FileSystem
PSIsContainer : False
FileName     : C:\House\house_log.txt
Stream       : :$DATA
Length       : 13
```

```
PSPath      : Microsoft.PowerShell.Core\FileSystem::C:\House
\house_log.txt:housinginfo
PSParentPath : Microsoft.PowerShell.Core\FileSystem::C:\House
PSChildName  : house_log.txt:housinginfo
PSDrive      : C
PSProvider   : Microsoft.PowerShell.Core\FileSystem
```

```
PSIsContainer : False
FileName     : C:\House\house_log.txt
Stream       : housinginfo
Length       : 21
```

In the output above, you can see two streams that are attached to the file `C:\House\house_log.txt`:

1. `:$DATA` is the default data stream of every NTFS file. It contains the normal file contents and is not an ADS.
2. `housinginfo` is the Alternate Data Stream (ADS) added to this file. It appears as `house_log.txt:housinginfo`, which means an extra hidden stream named `housinginfo` is attached to this file.

Answer the questions below

In the previous task, you found a marvellous treasure carefully hidden in the lab machine. What is the hash of the file that contains it?

71FC5EC11C2497A32F8F08E61399687D90ABE6E204D2964DF589543A613F3E08

What property retrieved by default by `Get-NetTCPConnection` contains information about the process that has started the connection? `OwningProcess`

Task 7 - Scripting

Scripting is the process of automating tasks by writing commands in a script file.

PowerShell scripts are widely used in cybersecurity for automation, system management, and security operations.

- **Blue teams** use PowerShell for log analysis, threat hunting, detecting anomalies, extracting IOCs, malware analysis, and intrusion detection.
- **Red teams** use PowerShell for system enumeration, remote command execution, penetration testing, and simulating real-world attacks.
- **System administrators** use PowerShell to automate configurations, enforce security policies, monitor system health, and manage large environments.

Due to its deep integration with Windows systems, PowerShell is a powerful tool for both attackers and defenders, making monitoring and understanding PowerShell activity essential in cybersecurity.

Terminal

```
PS C:\Users\captain> Get-Help Invoke-Command -examples
```

NAME

Invoke-Command

SYNOPSIS

Runs commands on local and remote computers.

`Invoke-Command` is a PowerShell cmdlet used to **execute commands or scripts on remote computers**.

- It can run custom scripts remotely, making it useful for automating administrative and security tasks across multiple systems.
- It can also execute simple commands without requiring scripting knowledge by using the `ScriptBlock { }` parameter.
- Commands run through `Invoke-Command` behave as if they were executed directly in a PowerShell session on the remote machine.

Example:

```
Invoke-Command-ComputerNameServer01-ScriptBlock {Get-Process }
```

From a cybersecurity perspective, defenders use it for remote administration and incident response, while attackers may abuse it for remote command execution after gaining access.

Answer the questions below

What is the syntax to execute the command Get-Service on a remote computer named "RoyalFortune"?

Assume you don't need to provide credentials to establish the connection. [for the sake of this question, avoid the use of quotes (" or ') in your answer] `Invoke-Command -ComputerName RoyalFortune -ScriptBlock { Get-Service }`